

R.M IMOBILIÁRIA

Orçamento de Aluguel

Parte Teórica Fluxograma e Estrutura Lógica

1. Objetivo da aplicação

A aplicação "Orçamento de Aluguel" automatiza a geração de orçamentos de locação para a imobiliária R.M, que trabalha com três tipos de imóvel: Apartamento, Casa e Estúdio cada um com suas próprias regras de valores adicionais e descontos. O programa coleta os dados do cliente e do imóvel escolhido, aplica as regras de negócio correspondentes, calcula o valor final do orçamento (aluguel + contrato) e gera um arquivo CSV com o cronograma de 12 parcelas.

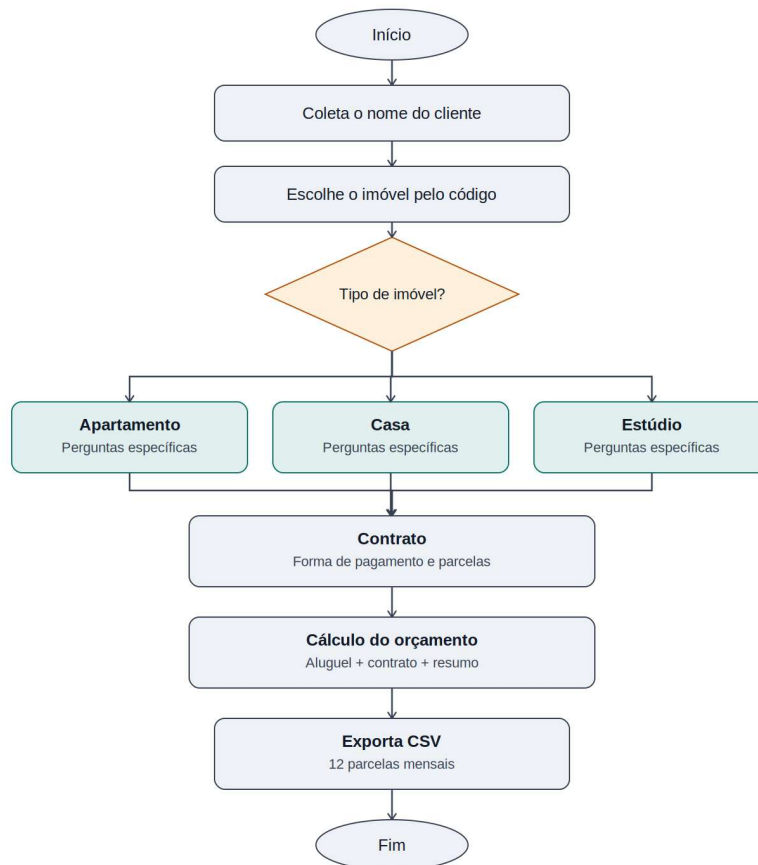
2. Pensamento algorítmico aplicado

A lógica do programa segue uma sequência linear com pontos de decisão e repetição, típica do pensamento algorítmico estruturado:

- Sequência: os passos ocorrem em ordem fixa: coleta de dados → cálculo → apresentação do resultado.
- Decisão (seleção): o tipo de imóvel escolhido direciona o fluxo para um conjunto diferente de perguntas e regras de cálculo (estrutura condicional).
- Repetição (iteração): toda entrada do usuário passa por um laço de validação que repete a pergunta até receber uma resposta válida por exemplo, código de imóvel, resposta Sim/Não, forma de pagamento e número de parcelas.
- Abstração e reuso: as regras de cálculo de cada tipo de imóvel foram isoladas em funções/métodos próprios, evitando repetição de código e facilitando manutenção.

3. Fluxograma da aplicação

O fluxograma abaixo representa o fluxo principal do programa, da coleta do nome do cliente até a geração do arquivo CSV com as parcelas.



Observação: dentro de cada ramo "Perguntas específicas" (Apartamento / Casa / Estúdio), o programa faz uma sequência própria de decisões Sim/Não, detalhadas na seção 4 e nos Anexos A.

4. Regras de negócio por tipo de imóvel

4.1 Apartamento por R\$ 700,00 (base, 1 quarto)

- 2 quartos? → Sim: +R\$ 200,00 na mensalidade
- Vaga de garagem? → Sim: +R\$ 300,00 na mensalidade
- Possui criança? → Não: desconto de 5% sobre o subtotal (base + adicionais)

4.2 Casa por R\$ 900,00 (base, 1 quarto)

- 2 quartos? → Sim: +R\$ 250,00 na mensalidade
- Vaga de garagem? → Sim: +R\$ 300,00 na mensalidade

4.3 Estúdio por R\$ 1.200,00

- Estacionamento? → Sim: +R\$ 250,00 (já inclui 2 vagas)
- Vagas extras? → +R\$ 60,00 por vaga adicional

4.4 Contrato imobiliário

- Valor fixo de R\$ 2.000,00, parcelado em até 5 vezes
- O aluguel mensal é pago à parte; a parcela do contrato se soma ao aluguel apenas durante os meses do parcelamento
- Ao final, é gerado um arquivo **.csv** com o cronograma das 12 parcelas mensais, com datas calculadas a partir da data de geração do orçamento

5. Estrutura orientada a objetos

O código foi estruturado usando os principais pilares da Programação Orientada a Objetos:

Conceito	Onde aparece no código
Herança	As classes Apartamento, Casa e Estúdio herdam de uma classe base Imóvel, reaproveitando atributos e o método valor_mensal().
Polimorfismo	Cada subclasse implementa perguntar_opcionais() à sua maneira; o programa principal chama esse método sem precisar saber qual classe está por trás.
Encapsulamento	Cada classe guarda seus próprios dados (adicional_quartos, desconto etc.) e constantes (ADICIONAL_QUARTO, DESCONTO_SEM_CRIANCA).
Composição	A classe Orçamento não herda de Imóvel nem de Contrato — ela os contém como atributos, combinando os dois para gerar o resumo e o CSV.

6. Pseudocódigo resumido

```
INÍCIO
  cliente ← LER("Nome do cliente")
  MOSTRAR lista de imóveis disponíveis
  REPITA
    codigo ← LER("Código do imóvel")
    ATÉ codigo ser válido

    imovel ← CRIAR objeto do tipo correspondente ao código
              (Apartamento, Casa ou Estudio)
    imovel.PERGUNTAR_OPCIONAIS() // decisões específicas do tipo

  REPITA
    forma_pagamento ← LER("Forma de pagamento")
    ATÉ forma_pagamento ser válida

  REPITA
    n_parcelas ← LER("Número de parcelas (1 a 5)")
    ATÉ n_parcelas estar entre 1 e 5

  valor_parcela ← 2000 / n_parcelas
  valor_mensal ← imovel.valor_base + adicionais - desconto

  MOSTRAR resumo do orçamento
  GERAR arquivo CSV com 12 parcelas
```

7. Trechos de código representativos

O código completo está no arquivo `Orçamento_de_Aluguel.py`, entregue na Parte Prática. Abaixo, os trechos que implementam diretamente a lógica descrita nas seções anteriores.

7.1 Hierarquia de classes (herança e polimorfismo item 5)

A classe base `Imovel` define o comportamento comum; cada subclasse implementa `perguntar_opcionais()` com suas próprias regras (seção 4):

```
class Imovel:
    def __init__(self, codigo, localizacao, metragem, banheiros,
valor_base):
        self.codigo = codigo
        ...
        self.adicional_quartos = 0.0
        self.adicional_garagem = 0.0
        self.desconto = 0.0

    def perguntar_opcionais(self):
        raise NotImplementedError # cada subclasse define o seu
    def valor_mensal(self):
        return self.valor_base + self.adicional_quartos \
            + self.adicional_garagem - self.desconto

class Apartamento(Imovel):
    ADICIONAL_QUARTO = 200.00
    ADICIONAL_GARAGEM = 300.00
    DESCONTO_SEM_CRIANCA = 0.05

    def perguntar_opcionais(self):
        if perguntar_sn('Deseja 2 Quartos? (S) Sim | (N) Não: ') == 's':
            self.quartos = '2'
            self.adicional_quartos = self.ADICIONAL_QUARTO
        # ... (vaga de garagem e desconto sem criança seguem a mesma
lógica)
```

7.2 Fluxo principal (corresponde ao Anexo A)

A função `main()` segue exatamente a sequência do fluxograma: coleta → escolha do imóvel → cria o objeto certo → perguntas específicas → contrato → cálculo → saída:

```
def main():
    cliente = input('Digite o nome do cliente: ')

    for codigo, (classe, ...) in CADASTRO_IMOVEIS.items():
        print(f'{codigo} - {classe.__name__} | ...')
```

```

    codigo_imovel = input('\nDigite o código do imóvel desejado:
').strip().upper()
    while codigo_imovel not in CADASTRO_IMOVEIS:
        print('Código de imóvel inválido! Tente novamente.')
        codigo_imovel = input(...).strip().upper()

    classe, localizacao, metragem, banheiros, valor_base =
CADASTRO_IMOVEIS[codigo_imovel]
    imovel = classe(codigo_imovel, localizacao, metragem, banheiros,
valor_base)
    imovel.perguntar_opcionais()    # polimorfismo: cada tipo pergunta o
que precisa

    contrato = Contrato()
    contrato.escolher_forma_pagamento()
    contrato.escolher_parcelamento()

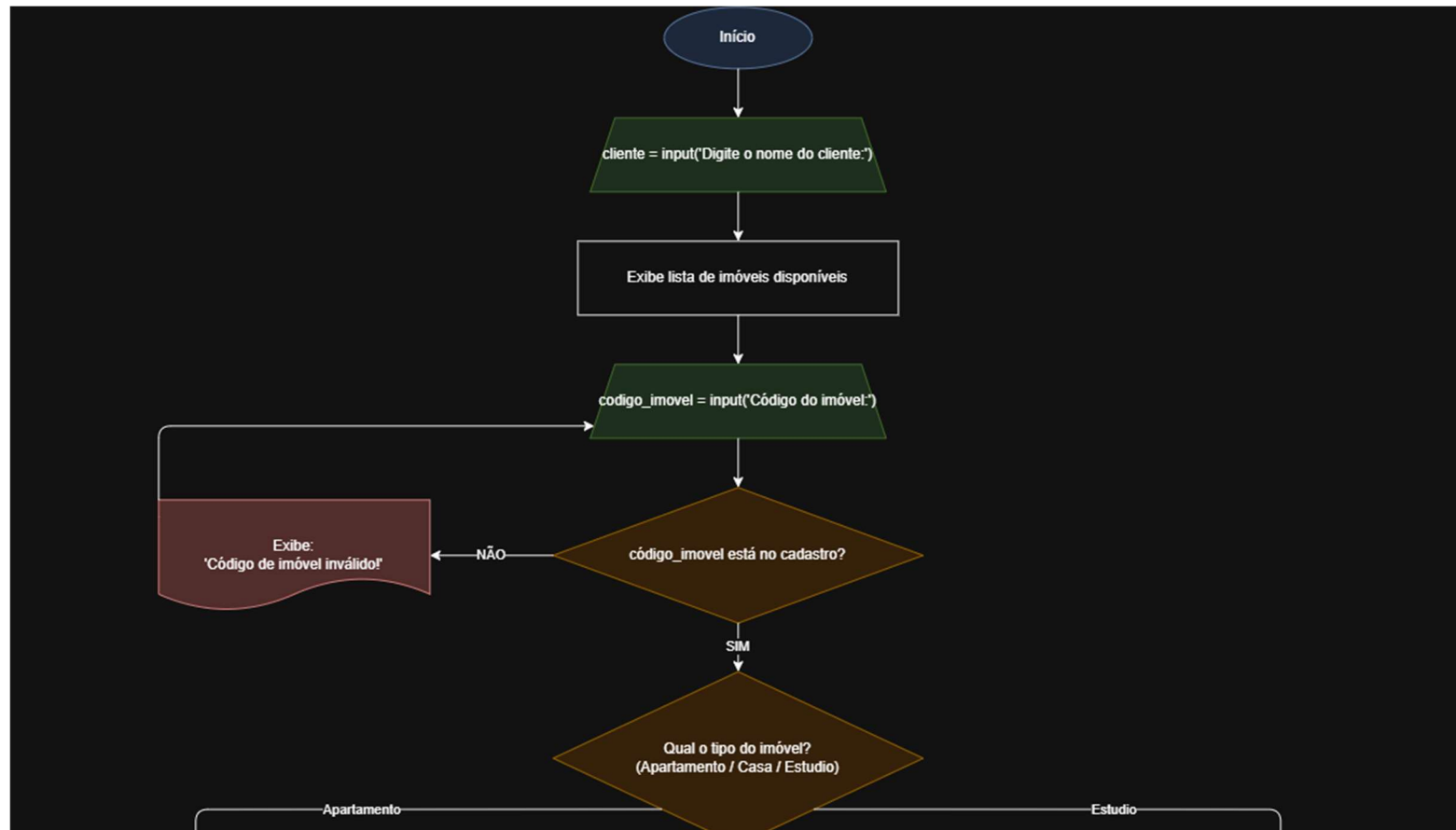
    orcamento = Orcamento(cliente, imovel, contrato)
    orcamento.exibir_resumo()
    orcamento.exportar_csv()

```

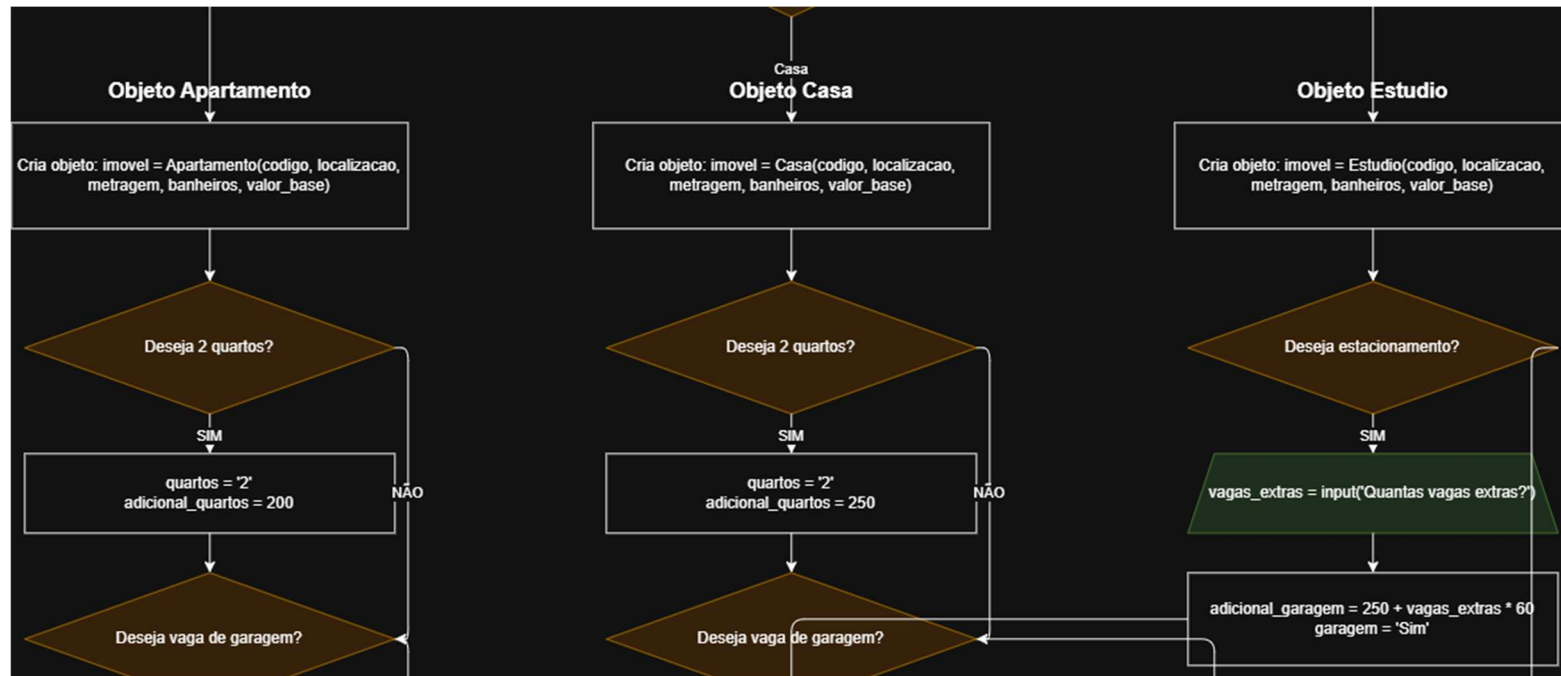
8. Conclusão

A estrutura do programa aplica sequência, decisão e repetição de forma clara, e a modelagem orientada a objetos (herança e polimorfismo entre Imóvel, Apartamento, Casa e Estúdio) evita duplicação de código e torna o sistema fácil de estender por exemplo, adicionar um novo tipo de imóvel exigiria apenas criar mais uma subclasse, sem alterar a lógica principal do programa.

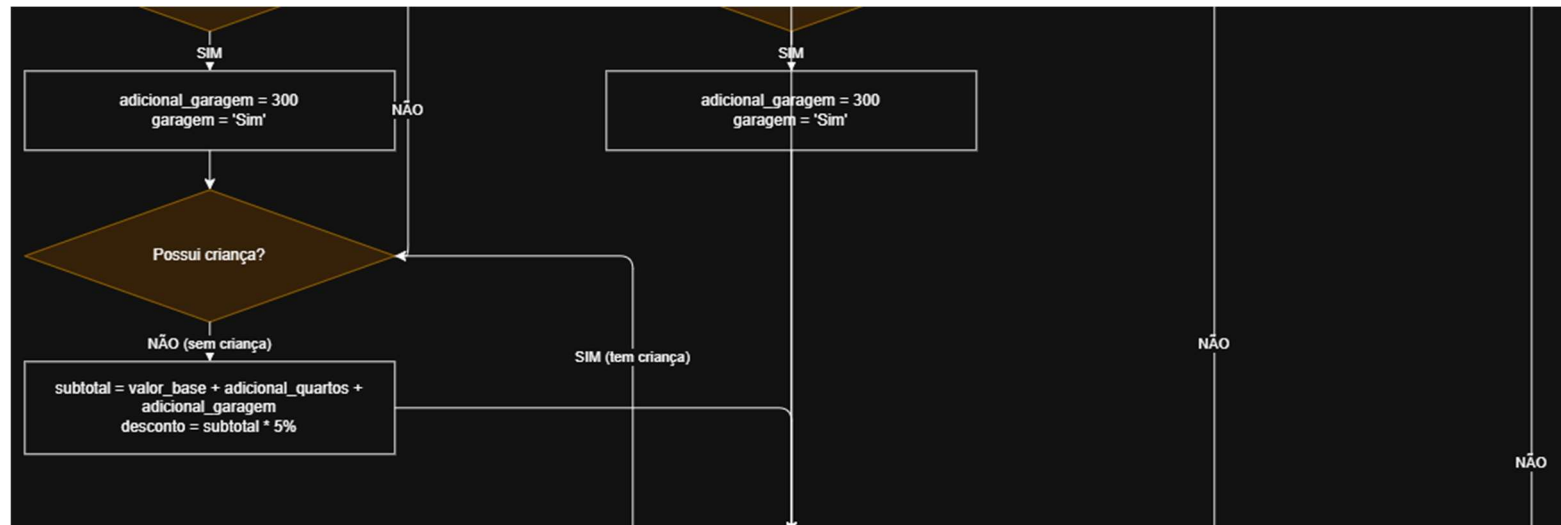
Anexo A (1/5) Início, escolha do imóvel e tipo



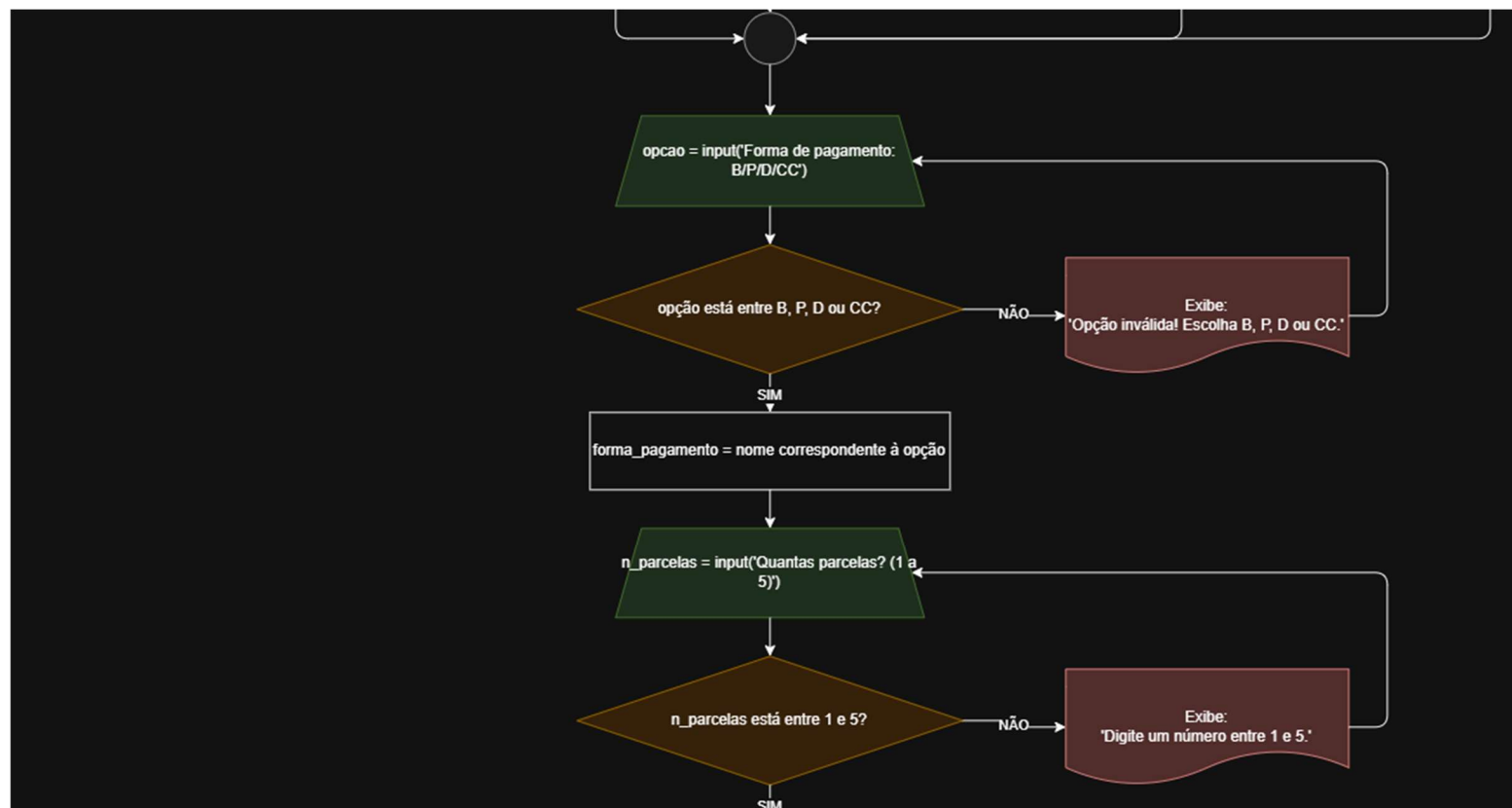
Anexo A (2/5) Perguntas específicas de cada tipo (parte 1)



Anexo A (3/5) Perguntas específicas de cada tipo (parte 2) e junção dos fluxos



Anexo A (4/5) Forma de pagamento e parcelamento do contrato



Anexo A (5/5) Cálculo final, resumo, exportação do CSV e fim

